

Module 9: Secure Data Storage

Class Discussion Deck — Session 1 & Session 2

File Permissions · Encryption · Cloud

Real-World Cases

Java · JS · React

Every code sample includes a line-by-line walkthrough — what it does, how it works, and how to read it.

Two 3-hour class sessions, held on two different days.

DAY 1 · 3 HOURS

SESSION 1

Storage Security Fundamentals

- The CIA Triad
- Local Storage Controls (+ Hands-On Lab)
- Database & Cloud Storage + Misconfigurations
- Real-World Example: The 2016 Uber Breach
- Code Demos: Java, JavaScript & React

The CIA Triad for Data at Rest

Goal	Means	Storage Example
Confidentiality	Only authorized parties can read it	Encrypting a database column
Integrity	Data hasn't been secretly altered	Checksums that detect tampering
Availability	Users can actually get it when needed	Backups and redundancy

LAYMAN'S ANALOGY — THE TREASURE CHEST WITH THREE LOCKS

Confidentiality is a solid lid so no one can peek inside without the key. Integrity is a wax seal across the lid's edge — if broken, you know someone tampered with the contents. Availability is making sure the chest actually opens when the rightful owner needs it — a chest rusted shut forever has failed just as badly as one with no lock at all.

Local Storage Controls

- File ownership and permissions (read / write / execute).
- Least privilege: give every file only the exact access it needs.
- Secure defaults — never "everyone can do everything" for convenience.

PREVIEW

Today we type real `chmod` commands to build `public.txt`, `internal.txt`, and `sensitive.txt` — see next slide.

LAYMAN'S ANALOGY — THE SHARED APARTMENT

The corkboard in the hallway is public — any roommate or visitor can read it. The shared binder in the kitchen is internal — only roommates can flip through it.

The locked drawer in your own room is sensitive — only you hold that key. This is exactly the structure of `public.txt` / `internal.txt` / `sensitive.txt`.



Building the SECURE_STORAGE_LAB (macOS/Linux)

```
1  mkdir SECURE_STORAGE_LAB
2  cd SECURE_STORAGE_LAB
3  echo "Public info" > public.txt
4  echo "Internal use only" > internal.txt
5  echo "Sensitive data" > sensitive.txt
6  chmod 644 public.txt
7  chmod 640 internal.txt
8  chmod 600 sensitive.txt
```

RESULT

```
public.txt    → readable by everyone
internal.txt  → owner + group only
sensitive.txt → owner only, no one else
```

HOW TO READ THIS — LINE BY LINE

- 1 Creates a brand-new folder to hold our three test files.
- 2 Moves into that folder so every command after this happens inside it.
- 3-5 Creates three simple text files, one for each sensitivity level.
- 6 644 = owner can read/write, everyone else can only read — fine for public info.
- 7 640 = owner can read/write, the group can only read, outsiders get nothing.
- 8 600 = owner only — no one else on this machine can even read this file.

Database & Cloud Storage + Misconfigurations

- Database controls: encrypted disks, column encryption, strong auth, protected backups.
- Cloud: shared responsibility, IAM roles, security groups, managed encryption (KMS).
- Common misconfigurations: public buckets, broad IAM policies, open DB ports, weak admin accounts, unencrypted backups.

QUICK CHECK

A misconfigured public bucket can hand data to anyone without them ever needing an encryption key at all.

LAYMAN'S ANALOGY — RENTING A STORAGE UNIT

Using a cloud provider is like renting a unit in a guarded facility: the company secures the building and the perimeter — that's their half of "shared responsibility."

But YOU still have to lock your own unit door. A cloud misconfiguration is exactly like renting that secure unit and leaving your own door wide open.



The 2016 Uber Breach

57 Million Records — Because a Key Was Hiding in Plain Code

- In September 2016, attackers gained access to a private GitHub repository used by Uber engineers and found an AWS access key sitting in plaintext, hardcoded directly in the source code.
- That single key gave them access to an S3 bucket holding rider and driver data. They downloaded records for approximately 57 million users worldwide, plus license numbers for 600,000 U.S. drivers.
- Uber's then-CSO paid the attackers \$100,000 to delete the data and stay silent — the breach wasn't disclosed for over a year.
- When it came to light in 2017, Uber paid a \$148 million settlement, and in 2022 that former CSO was criminally convicted of obstruction of justice for the cover-up.



Encrypting Data Before It's Stored

JAVA — ENCRYPTION-AT-REST

```
1 KeyGenerator keyGen = KeyGenerator.getInstance("AES");
2 keyGen.init(128);
3 SecretKey secretKey = keyGen.generateKey();
4 String data = "Juan Dela Cruz - Pass...";
5 Cipher cipher = Cipher.getInstance("AES");
6 cipher.init(ENCRYPT_MODE, secretKey);
7 byte[] encrypted = cipher.doFinal(data.getBytes());
8 cipher.init(DECRYPT_MODE, secretKey);
9 byte[] decrypted = cipher.doFinal(encrypted);
```

CONSOLE OUTPUT

```
Stored: (long unreadable string)
Read: Juan Dela Cruz - Password: myPass123
```

HOW TO READ THIS — LINE BY LINE

1-3 Creates a random AES secret key — the "lock and key" that will protect our data.

4 The real, sensitive data we want to store safely.

5-6 Sets up the cipher tool in ENCRYPT mode, using our key.

7 Scrambles the data into unreadable bytes — this is what actually gets saved to disk.

8-9 Switches the SAME cipher to DECRYPT mode and reverses the scrambling.



Never Hardcode a Password

JAVA — SECRETS: BAD VS. GOOD

```
1 String dbPassword = "SuperSecret123";  
2 String dbPassword = System.getenv("DB_PASSWORD");
```

HOW TO READ THIS — LINE BY LINE

- 1 BAD:** the real password is typed directly into the source code — anyone who reads this file (or finds it on GitHub, like in the Uber breach) instantly has it.
- 2 GOOD:** the password is never written in the file at all — it's read from an environment variable set outside the code, at deploy time.



The Same Idea, in JavaScript

JAVASCRIPT (NODE.JS) — ENCRYPTION

```
1  const key = crypto.randomBytes(32);
2  const iv = crypto.randomBytes(16);
3  function encryptData(text) {
4    const cipher = crypto.createCipheriv(...);
5    let encrypted = cipher.update(text, 'utf8', 'hex');
6    encrypted += cipher.final('hex');
7    return encrypted;
8  }
```

CONSOLE OUTPUT

```
Stored: (long hexadecimal string)
Read:   Juan Dela Cruz - Password: myPass123
```

HOW TO READ THIS — LINE BY LINE

- 1** Generates a random 32-byte encryption key (AES-256).
- 2** Generates a random "initialization vector" — an extra safeguard against patterns.
- 3-4** Defines a function that builds an AES cipher using our key and iv.
- 5-6** Feeds the real text through the cipher, producing scrambled hex output.
- 7-8** Returns the scrambled result — this is what gets saved, never the original text.



Loading Secrets From the Environment

JAVASCRIPT — SECRETS: BAD VS. GOOD

```
1  const apiKey = "sk_live_51H3xamplekey";  
2  require('dotenv').config();  
3  const apiKey = process.env.API_KEY;
```

HOW TO READ THIS — LINE BY LINE

- 1** BAD: the real API key is typed directly into the file — visible to anyone who opens it or finds it on GitHub.
- 2** GOOD: loads variables from a hidden .env file that is never committed to version control.
- 3** The key is read from that environment variable instead — the actual value never appears in this file.



Front-End Code Can Never Keep a Secret

REACTJS — WHY THE FRONT END IS DIFFERENT

```
1  localStorage.setItem("authToken", token);  
2  fetch("/api/login", { method: "POST", credentials: "include" });  
3  const PUBLISHABLE_KEY = process.env.REACT_APP_STRIPE_PUBLISHABLE_KEY;
```

HOW TO READ THIS — LINE BY LINE

- 1 VULNERABLE:** saves the token in the browser's local storage, where any script on the page (including malicious ones) can read it instantly.
- 2 SECURE PATTERN:** the server sets the token as an HttpOnly cookie instead — React's own JavaScript can never read it at all.
- 3** Only PUBLIC/publishable keys belong in React code — never a secret key, since anyone can view the page's source.

What We Covered Today

- 1 The CIA Triad**
Confidentiality, Integrity, Availability — the three storage security goals.
- 2 Local Storage Controls**
File permissions and least privilege, typed as real chmod commands.
- 3 DB/Cloud Storage + Misconfigs**
Shared responsibility — the provider secures the building, you lock your unit.
- 4 Java, JS & React Code Demos**
Encryption-at-rest and "never hardcode secrets," in all three languages.

DAY 2 · 3 HOURS

SESSION 2

From Local Files to the Cloud

- The Calibration Checklist
- Real-World Example: The 2017 Verizon/NICE Breach
- Reading a Real AWS S3 Storage Policy

The Calibration Checklist

- Access is based on roles (least privilege), not convenience.
- Sensitive fields are encrypted and keys are protected.
- Backups exist and are protected (encrypted + restricted).
- Public access is disabled unless explicitly required and justified.
- Audit/logging is enabled for storage access and changes.

KEY PHRASE TO REMEMBER

"...not convenience." The most common real-world excuse for a misconfiguration is loosening a permission temporarily to make testing easier — then forgetting to tighten it before shipping.



The 2017 Verizon / NICE Systems Breach

Up to 14 Million Records — From One Publicly-Open Cloud Folder

- In June 2017, researcher Chris Vickery discovered an Amazon S3 bucket configured to allow full public access — anyone who found its URL could download the entire contents.
- The bucket belonged to NICE Systems, a vendor handling Verizon's call-center operations. Inside were customer names, addresses, account details, and account PINs.
- No password was cracked. No encryption was broken. The data was sitting in a folder that had simply been set to public instead of private.
- Which single Calibration Checklist line would have caught this? "Public access is disabled unless explicitly required and justified" — one setting, checked once, prevents the entire incident.



Reading a Cloud Storage Policy Like an Auditor

AWS S3 POLICY — VULNERABLE

```
1  "Version": "2012-10-17",
2  "Statement": [{
3    "Effect": "Allow",
4    "Principal": "*",
5    "Action": "s3:GetObject",
6    "Resource": ".../*"
7  }]
```

HOW TO READ THIS — LINE BY LINE

- 1-2** Standard header identifying this as an access policy document.
- 3** This rule **ALLOWS** the action described below — nothing is blocked here.
- 4** "*" means literally anyone on the internet, with no login required at all.
- 5** The allowed action is reading (GetObject) — downloading a file.
- 6-7** The wildcard /* means every single file in the entire bucket, no exceptions.



The Fix: Least Privilege in Real Cloud Syntax

AWS S3 POLICY — CORRECTED

```
1  "Principal": { "AWS": "arn:aws:iam::123456789012:role/
    AppBackendRole" },
2  "Action": "s3:GetObject",
3  "Resource": "arn:aws:s3:::...uploads/public/*" ,
4  "Condition": { "Bool": { "aws:SecureTransport": "true" } }
```

HOW TO READ THIS — LINE BY LINE

- 1 Now only ONE specific, named application role is allowed — not the entire world.
- 2 Still only allows reading, never writing or deleting.
- 3 Scoped down to only the /public/ folder — not the whole bucket.
- 4 Requires an encrypted (HTTPS) connection even for this narrow, limited access.

What We Covered Today

5

The Calibration Checklist

Roles, encrypted fields, protected backups, disabled public access, logging.

—

Real-World Example

Verizon/NICE Systems — 14 million records from one public bucket setting.

—

Reading a Cloud Policy

A real AWS S3 policy, vulnerable vs. corrected, line by line.

MODULE 9 SUMMARY

Secure Data Storage

Local Permissions

the labeled drawers in your shared apartment — `chmod`, typed for real.

Encryption-at-Rest

the locked box inside the drawer, in Java, JavaScript, and beyond.

Cloud Misconfigurations

rarely a broken lock — usually a door somebody left open.

NEXT MODULE PREVIEW

Securing data while it's actively in use — session management, secure APIs, and monitoring/logging in depth.

References

- 1 Information Assurance and Security 1 course outline: Module 9 - Secure Data Storage.
- 2 Instructor/NetAcad readings and demos on OS file permissions and database security.
- 3 OWASP guidance/handouts on secure storage configuration and secrets management.
- 4 Uber data breach (September–November 2016, disclosed 2017) — AWS key hardcoded in a private GitHub repo; ~57 million records exposed; \$148 million multistate settlement.
- 5 Verizon / NICE Systems data exposure (discovered June 2017 by UpGuard) — a publicly accessible AWS S3 bucket exposed up to 14 million customer records.